Seminararbeit

Ludwig-Maximilians-Universität München
Fakultät für Mathematik, Informatik und Statistik
Seminar: Introduction to Survival Analysis

Survival Trees and Random Survival Forests

Name: Jakob Haas
Matrikelnummer: 12704317
Studiengang: Statistik und Data Science
Betreuer: Dr. Andreas Bender
Datum: 12. Juni 2026

Contents

1 Introduction

Survival analysis is one of the core areas of statistics. Its main objective is to analyze and predict the time until the occurrence of an event, such as death from a disease or the end of unemployment.
Standard statistical methods, such as regression, cannot be applied directly in this context, since survival data are typically subject to censoring.[1] To address this problem, classical statistics provides several methods, such as the non-parametric Kaplan–Meier estimator, the semi-parametric Cox model, and parametric survival models. Although these classical methods are widely used, they may be limited for example when the data exhibit nonlinear effects. This motivates the use of machine learning methods which are particularly strong in modeling nonlinear relationships and predictions (Wang, Li, and Reddy 2019, p. 18). Therefore, the standard machine learning workflow has to be adapted to the specific structure of right-censored survival data.

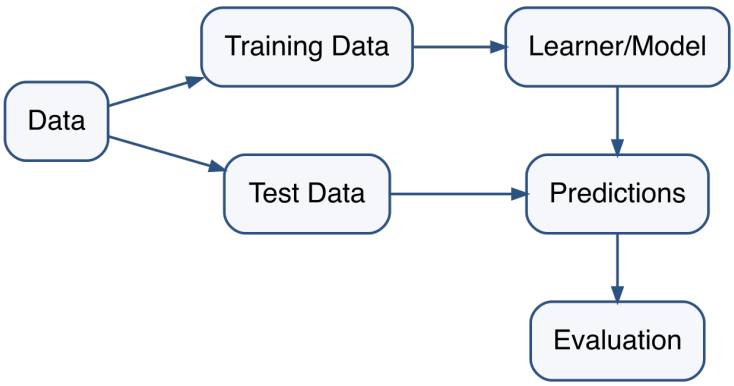

Figure 1: Machine learning workflow

In a standard supervised machine learning setting, as shown in Figure 1, the data is first split into a training set and a test set. The training set is used to fit the model and possibly to tune hyperparameters. The test set is kept separate and is only used at the end to evaluate the predictive performance on unseen data. The fitted model is then used to make predictions on the test data, and these predictions are evaluated using an appropriate performance measure. The separation of training data and test data is crucial to obtain an unbiased estimate of the model performance.
In the classical setup, each observation usually consists of a feature vector $\mathbf{x}_i$ and a fully observed outcome y_i. The goal is then to learn a function that predicts y_i from $\mathbf{x}_i$. In survival analysis, the response variable is more complex. Instead of observing the true event time for an individual i, we just observe:

$$T_i = \min(T_i^*, C_i),$$

where T_i^* denotes the true event time and C_i the censoring time. In addition, we observe the event indicator:

$$\delta_i = \mathbb{I}(T_i^* \leq C_i)$$

which indicates whether the event was observed or whether the observation was right-censored.
Consequently, learning algorithms for survival data must account for both the observed

[1] In this seminar paper, we always assume right-censored data. This means that we only know that an individual has not experienced the event up to a certain time point T_i.

time T_i and the censoring indicator δ_i. In addition, the prediction target changes. Instead of predicting a single event time, survival models typically estimate survival-related functions, such as the survival function or the cumulative hazard function. Finally, the evaluation procedure also has to be adapted.
This seminar paper focuses on tree-based machine learning methods for right-censored survival data. Survival trees extend classical decision trees by adapting the splitting rules and prediction targets to censored time-to-event outcomes. Random Survival Forests build on this idea by aggregating many survival trees into an ensemble, which increases stability and predictive accuracy.

2 Survival Trees

2.1 Classification and Regression Trees (CART)

Classification and Regression Trees (CART) are a standard method in machine learning. This method was introduced by Leo Breiman et al. (1984).[2] CART works as follows:

1. We start with a root node that contains all the data.
2. Then we search for the feature x_p and split that minimizes the empirical risk in the child nodes.
3. We do this recursively for each child node: We find the best split and then split the data into two child nodes.
4. We continue doing this until a stopping criterion is met. The final nodes are then called terminal nodes.

Figure 2 illustrates the concept of a classification tree using the `Iris` data from the `R` (R Core Team 2025) data base.

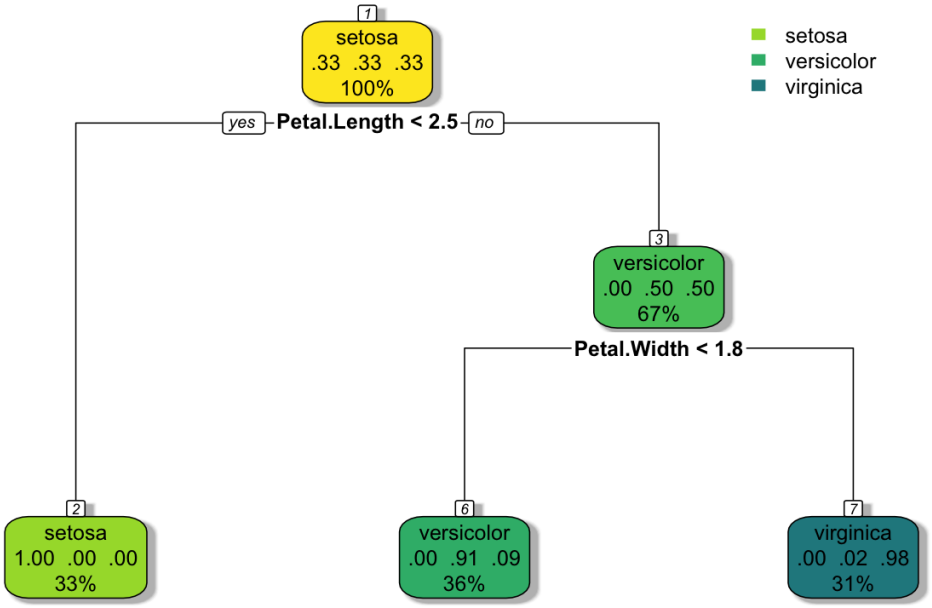

Figure 2: Example of a CART classification tree for the Iris data set.

[2]For a detailed introduction to tree-based methods, see James et al. (2013, chap. 8).

Predictions are then generated by sending a new observation with feature vector $\mathbf{x}$ down the tree until it reaches a terminal node. For classification problems, the predicted class is the majority class of training data in that terminal node, if a hard label prediction is desired. If we aim to predict class probabilities, then we use relative class frequencies within the terminal node. So the estimated probability for class k is given by:

$$\hat{P}[Y = k \mid \mathbf{x}] = \frac{1}{|R_m|} \sum_{i:\mathbf{x}_i \in R_m} \mathbb{I}(y_i = k),$$

where R_m denotes the terminal node into which the new observation $\mathbf{x}$ falls. For regression problems, the prediction is usually the average response of the training observations in the corresponding terminal node. Back to our tree from Figure 2. Imagine we have the new observation from Table 1: When we send this observation down the tree shown in

Table 1: New hypothetical observation for the Iris dataset

Feature	Sepal length	Sepal width	Petal length	Petal width
$\mathbf{x}$	5.1	3.5	2.8	0.2

Figure 2, it lands in terminal node 6. So the hard label prediction would be *versicolor* and the probabilities would be:

$$\hat{\mathbb{P}}[\text{species} = \text{setosa} \mid \mathbf{x}] = 0$$

$$\hat{\mathbb{P}}[\text{species} = \text{versicolor} \mid \mathbf{x}] = 0.91$$

$$\hat{\mathbb{P}}[\text{species} = \text{virginica} \mid \mathbf{x}] = 0.09.$$

2.2 From Decision Trees to Survival Trees

So far we have introduced decision trees for regression and classification problems. Now, we want to apply this model class to survival data. Throughout, survival data are taken to consist of observations of the form (T_i, δ_i), where T_i denotes the observed time for an individual i, and δ_i is the event indicator, with $\delta_i = 1$ if the event of interest is observed and $\delta_i = 0$ if the observation is censored. When covariates are present, the observations are written as $(T_i, \delta_i, \mathbf{x}_i)$, where $\mathbf{x}_i$ denotes the corresponding covariate vector.
A first natural idea would be to treat T_i as a continuous target and to fit a regression tree with T_i as the target variable. However, this approach is not sufficient for survival data, since T_i is not necessarily the true event time. Rather the observed time is given by:

$$T_i = \min(T_i^*, C_i),$$

where T_i^* denotes the true event time and C_i the censoring time. The event indicator is:

$$\delta_i = \mathbb{I}(T_i^* \leq C_i)$$

Thus, for a censored observation, i.e., when $\delta_i = 0$, the only information available is: $T_i^* > T_i$. We can illustrate the problem with a simple example. Consider the following two observations:

$$(T_1, \delta_1) = (100, 1), \qquad (T_2, \delta_2) = (100, 0)$$

In both cases the observed time is $T_i = 100$, $i \in \{1, 2\}$. However, the interpretation is fundamentally different. For the first individual the event occurred at time 100, so $T_i^* = 100$. The observation of the second individual is censored at time 100, so we only

know the event has not occurred up to time 100.
Recall that our proposed regression tree selects splits such that the target values within the resulting daughter nodes are as similar as possible. If we simply use the observed time T_i as the response variable, the regression tree would treat both observations as having the same target value. However, this ignores the fact that one observation is an observed event and the other represents only a lower bound for the true event time. As a result, the usual regression tree splitting criterion may produce misleading splits and fail to create nodes with similar survival behavior.
Therefore, survival trees require a splitting criterion that explicitly accounts for both event times and censoring.

2.3 Splitting Rules in Survival Trees

Now, we want to discuss censoring-aware splitting criteria. A split at a node h on a feature x has the form $x \leq c$ vs. $x > c$ for a numerical feature, where c is the threshold. For example, if we have a feature with values $1, 4, 4, 6, 8$, our possible thresholds would be $1, 4, 6$ and a split at threshold $c = 4$ would produce the two child nodes $\mathcal{N}_1 = \{1, 4, 4\}$ and $\mathcal{N}_2 = \{6, 8\}$. For a categorical feature, a split has the form $x \in A$ vs. $x \notin A$, where A is a subset of the possible categories of x. For example, if the feature is binary ($x \in \{0, 1\}$), A would be $\{0\}$.
In ordinary decision trees, splits are chosen such that the target values within each child node are as homogeneous as possible. Now, we want to transfer this to survival data. This means that we aim to have observations with similar survival behavior in the same node. Therefore, splits are selected to separate groups with different survival distributions. This brings us to the log-rank test introduced by Mantel et al. (1966), which tests the null hypothesis that two groups $g \in \{1, 2\}$, here the two child nodes, have the same survival function:

$$H_0 : S_1(t) = S_2(t) \quad \forall t$$

The test statistic of the test is (Ishwaran and Kogalur 2007, p. 26):

$$L(x, c) = \frac{\sum_{j=1}^{m} \left(d_{j,1} - n_{j,1} \frac{d_j}{n_j} \right)}{\sqrt{\sum_{j=1}^{m} \frac{n_{j,1}}{n_j} \left(1 - \frac{n_{j,1}}{n_j} \right) \left(\frac{n_j - d_j}{n_j - 1} \right) d_j}} = \frac{\sum_{j=1}^{m} (d_{j,1} - \mathbb{E}[d_{j,1}])}{\sqrt{\sum_{j=1}^{m} \mathrm{Var}[d_{j,1}]}}, \tag{1}$$

where

- $t_1 < \cdots < t_m$ denote the distinct event times
- $d_{j,g}$ denotes the number of events at time t_j in group g
- $d_j = d_{j,1} + d_{j,2}$ denotes the total number of events at time t_j
- $n_{j,g}$ denotes the number of individuals at risk immediately before t_j in group g
- $n_j = n_{j,1} + n_{j,2}$ denotes the total number of individuals at risk immediately before t_j
- x denotes the variable that is split into two groups
- c denotes the split point for a numerical feature, or A is the subset of categories for a categorical feature.

Large values of $|L(x, c)|$ provide evidence against the null hypothesis and therefore indicate that the two groups have different survival distributions. Since the goal of a survival tree

is to obtain terminal nodes with homogeneous survival behavior, a split should separate observations with different survival distributions as strongly as possible. Hence, we choose the feature and split point that maximize $|L(x, c)|$. More formally, we want to find x^* and c^* such that:

$$(x^*, c^*) = \arg\max_{x,c} |L(x, c)|$$

For a numerical feature, c denotes the split point. For a categorical feature, the split is defined by a subset A of categories.
We now want to illustrate this splitting procedure with an example. We take the `tumor` dataset, which is available in the `R`-package `pammtools` (Bender and Scheipl 2018). The first four observations can be found in Table 2.

Table 2: Head of pammtools::tumor dataset

days	status	charlson_score	age	sex	transfusion	complications	metastases	resection
579	0	2	58	female	yes	no	yes	no
1192	0	2	52	male	no	yes	yes	no
308	1	2	74	female	yes	no	yes	no
33	1	2	57	male	yes	yes	yes	yes

Here, `days` is the time-to-event variable T_i and `status` is the censoring indicator δ_i. For our example we consider the two binary features `transfusion` and `complications`. In both cases, we split the data into transfusion vs. no transfusion and complications vs. no complications, respectively, and then calculate the log-rank statistic from (1). Figure 3 shows the resulting survival curves and the values of the log-rank statistic for the two candidate splits. Since the log-rank statistic of `complications` is higher than the log-rank statistic of `transfusion`, we split the data at feature `complications`. That means we split the data into observations which had complications and observations which had no complications.[3]

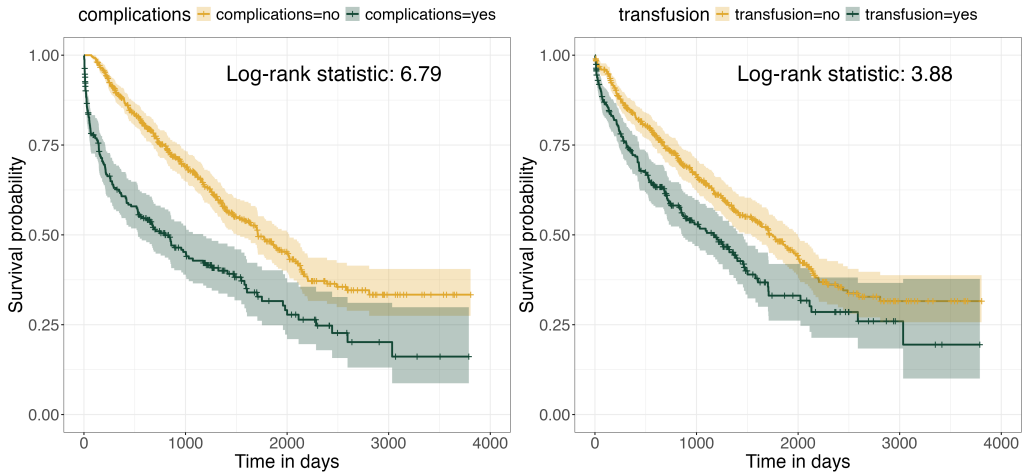

Figure 3: Possible log-rank splits for the tumor data

2.4 Prediction Targets

We now consider another aspect in which survival trees differ from ordinary decision trees. While ordinary decision trees typically predict a scalar response, survival trees estimate

[3]This split corresponds to the first split shown in Figure 4.

survival-related functions. One possible prediction target is the survival function:

$$S(t) = \mathbb{P}[T_i > t]$$

which describes the probability that an observation has not experienced the event up to time t.
In a survival tree, this function is estimated separately within each terminal node using the Kaplan-Meier estimator (Kaplan and Meier 1958). Thus, the prediction for terminal node h is given by:

$$\hat{S}_h(t) = \prod_{j:t_{j,h} \leq t} \left(1 - \frac{d_{j,h}}{n_{j,h}}\right) \tag{2}$$

where $d_{j,h}$ is the number of events at time $t_{j,h}$ and $n_{j,h}$ is the number of observations at risk immediately before $t_{j,h}$. Each terminal node h has its own ordered event times

$$t_{1,h} < t_{2,h} < \cdots < t_{m_h,h}.$$

Another possible prediction target is the cumulative hazard function (CHF):

$$H(t) = \int_0^t h(u)\, du,$$

where

$$h(u) = \lim_{\Delta t \to 0} \frac{\mathbb{P}[u \leq T^* < u + \Delta t \mid T^* \geq u]}{\Delta t}$$

is the hazard rate, which measures how likely an event is to occur immediately after time u, given that it has not occurred before u. So the CHF can be interpreted as the sum of those risks up to time t. Like the survival function, the CHF is estimated separately within each terminal node using the Nelson-Aalen estimator (Nelson 1972 and Aalen 1978). As a result, our prediction for terminal node h is given by:

$$\hat{H}_h(t) = \sum_{j:t_{j,h} \leq t} \frac{d_{j,h}}{n_{j,h}}. \tag{3}$$

Note that all observations that fall into the same terminal node receive the same prediction. If the prediction target is the survival function, they receive the same estimated survival function (2). If the prediction target is the CHF, they receive the same estimated CHF (3). This reflects the goal of obtaining groups with similar survival behavior within each terminal node.

2.5 Example for a Survival Tree

To conclude this section, we want to illustrate the derived method with an example. We consider the already known `tumor` data. We take the binary features `complications` and `transfusion` into account again. Additionally we fit the tree with the numeric feature `age`. The resulting tree can be seen in Figure 4.

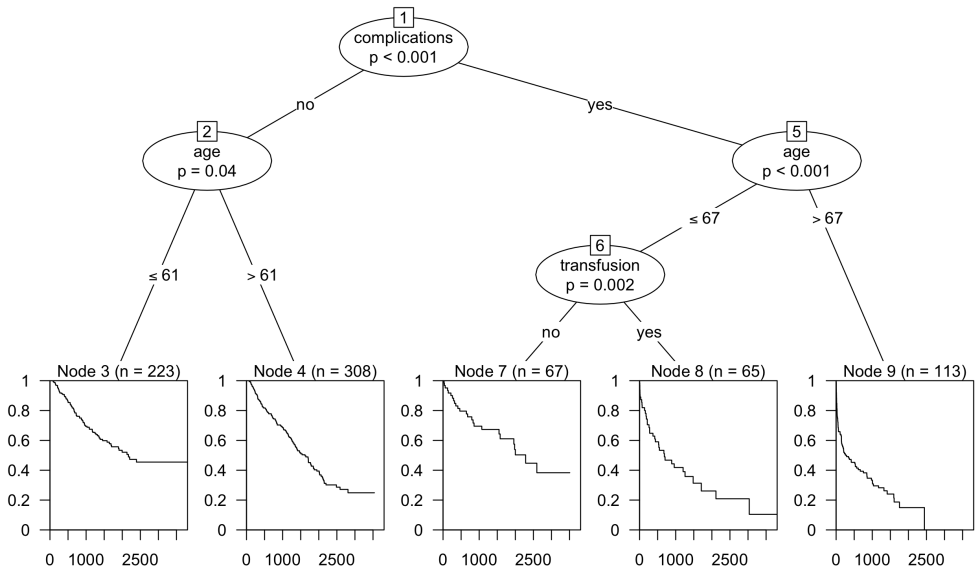

Figure 4: Example Survival Tree for the tumor data

As explained before, each terminal node has its own estimate of the survival function. Predictions work similarly to ordinary decision trees. Suppose that we have the following new observation:

complications	transfusion	age
yes	yes	64

We then drop this observation down the tree until it reaches node 8. The prediction for this observation is the survival function which we can see in node 8.

3 Random Survival Forests

3.1 From Survival Trees to Ensembles

In the previous section, we derived the construction and prediction of a single survival tree. This method has several advantages. For example, survival trees are easy to interpret and visualize and can capture interaction effects between features through their recursive splitting structure. However, single survival trees also have their limitations. One limitation that we want to address in this section is their instability. In this context, instability means that small changes in the training data can lead to substantially different tree structures. As a consequence, the resulting predictions may be sensitive to the particular training sample and therefore less reliable.
In this section, we introduce a method that addresses this issue by growing many randomized trees. Randomization is introduced in two ways. First, for each tree, a bootstrap sample is drawn from the training data. Bootstrapping is a resampling method in which a new dataset of size n is generated by sampling n observations from the original dataset with replacement. Thus, each tree is grown on a different bootstrap sample. Second, additional randomization is introduced through random feature selection. At each split, only a predefined number of features is considered as possible splitting candidates. This reduces the correlation between the individual trees, since different trees may use different variables for their splits. By combining many such randomized trees, the overall prediction becomes more stable than the prediction of a single survival tree.

3.2 Algorithmic structure

A Random Survival Forest can be trained using the following algorithm. For the algorithm we follow Ishwaran, Kogalur, et al. (2008). The inputs of our algorithm are:

- $\mathcal{D}_{\text{train}}$: the training data with the survival outcomes (T_i, δ_i) and p features $(x_1, ..., x_p)$
- B: the number of bootstrap samples drawn from $\mathcal{D}_{\text{train}}$ or the number of grown trees
- $\texttt{mtry} \leq p$: the number of features considered at each split
- $d_0 \geq 0$: minimum number of distinct events required in a terminal node.

B, `mtry` and d_0 are hyperparameters which we have to specify before running the algorithm. This can be done by hyperparameter tuning.[4] d_0 is the stopping criterion for growing the trees. If a further split of node $\mathcal{N}$ would result in at least one child node containing fewer than d_0 unique event times, the split is not performed and $\mathcal{N}$ becomes a terminal node. The algorithm then proceeds as follows (Ishwaran, Kogalur, et al. 2008, p. 4):

1. Draw B bootstrap samples from the data. Each bootstrap sample contains n observations with approximately 63.2% unique observations. The remaining observations are called out-of-bag data (OOB). [5]
2. Grow one survival tree from each bootstrap sample. At each node, randomly select `mtry` candidate features and choose the best split among them. Grow the tree under condition that each node must have at least d_0 unique event times.
3. Aggregate the terminal node predictions across all trees.

3.3 Aggregation of Survival Predictions

In subsection 2.4 we discussed prediction targets of survival trees. Now, we want to transfer this to Random Survival Forests. Remember, instead of predicting a scalar response variable, we predict a survival-related function. In Random Survival Forests we do the same with the so-called ensemble predictions. This works as follows:

- Imagine we have a new observation $\tilde{\mathbf{x}}$
- We push $\tilde{\mathbf{x}}$ down all our trained B trees
- In each tree $\tilde{\mathbf{x}}$ reaches one terminal node
- Every terminal node of the B trees has its own prediction $\hat{E}_b(t \mid \tilde{\mathbf{x}})$ which can be either the survival function or the cumulative hazard function.
- Finally we average point-wise the predicted functions of the terminal nodes to obtain the ensemble prediction:

$$\hat{E}_{ens}(t \mid \tilde{\mathbf{x}}) = \frac{1}{B} \sum_{b=1}^{B} \hat{E}_b(t \mid \tilde{\mathbf{x}}) \tag{4}$$

[4] For a detailed discussion of this topic see Yang and Shami (2020)

[5] The probability for an observation to be OOB in a tree is $\left(1 - \frac{1}{n}\right)^n$. For large n we get $\left(1 - \frac{1}{n}\right)^n \approx \frac{1}{e} \approx 0.368$. So the number of unique observations is approximately $1 - 0.368 = 0.632$.

For example, if our prediction target is the cumulative hazard function, then (4) becomes:

$$\hat{H}_{\text{ens}}(t \mid \tilde{\mathbf{x}}) = \frac{1}{B} \sum_{b=1}^{B} \hat{H}_b(t \mid \tilde{\mathbf{x}}),$$

where $\hat{H}_b(t \mid \tilde{\mathbf{x}})$ denotes the cumulative hazard function estimated in the terminal node into which $\tilde{\mathbf{x}}$ falls in the b-th tree.

3.4 Out-of-bag observations

As described in subsection 3.2, each tree in a Random Survival Forest is trained on a bootstrap sample of the original data. In each sample approximately $100\% - 63.2\% = 36.8\%$ of the observations are excluded. These left-out observations are called out-of-bag (OOB) observations. Thus, for each tree, the OOB observations were not used during the training process of the tree. They can therefore be used as internal validation data for this particular tree. The OOB observations are mainly used for two purposes.

OOB prediction error We can evaluate the prediction performance of our Random Survival Forest with our untouched OOB observations (Ishwaran, Kogalur, et al. 2008, p. 7). For each observation, predictions are obtained only from those trees for which the observation was out-of-bag. These predictions are then compared with the observed survival outcome using an appropriate performance measure (See section 4). The remarkable advantage of the OOB prediction error is that we have an untouched validation set without splitting the data.

Variable Importance Based on the OOB observations a Random Survival Forest provides a prediction-based measure of variable importance (Ishwaran, Kogalur, et al. 2008, p. 9). For a feature x, we can calculate its variable importance as follows:

1. Obtain the original OOB prediction error from the forest, as described before.
2. Push each observation down its corresponding OOB trees. Whenever a split on x is encountered, randomly assign the observation to one of the daughter nodes.
3. Compute the prediction error of this perturbed OOB ensemble.
4. Define the variable importance as:

$$\text{VIMP}(x) = \text{Err}_{\text{perturbed}}(x) - \text{Err}_{\text{OOB}}.$$

The idea is to remove the predictive information contained in x from the forest and then evaluate how much the prediction performance changes. The larger $\text{VIMP}(x)$ is, the stronger the contribution of x to the predictive accuracy of the forest. If $\text{VIMP}(x)$ is close to zero, perturbing x has little effect on the prediction error, which indicates that x has little predictive relevance for the fitted forest.
Variable importance values were calculated for all features of the `tumor` data set. The results are displayed in Figure 5. We can see that `complications` has the highest variable importance among all features. The two features `resection` and `sex` have values close to zero, meaning that perturbing these variables has little impact on the prediction error. The feature `charlson_score` has a negative variable importance. This indicates that perturbing this variable does not increase the OOB prediction error and may even slightly improve the predictive performance of the fitted forest.

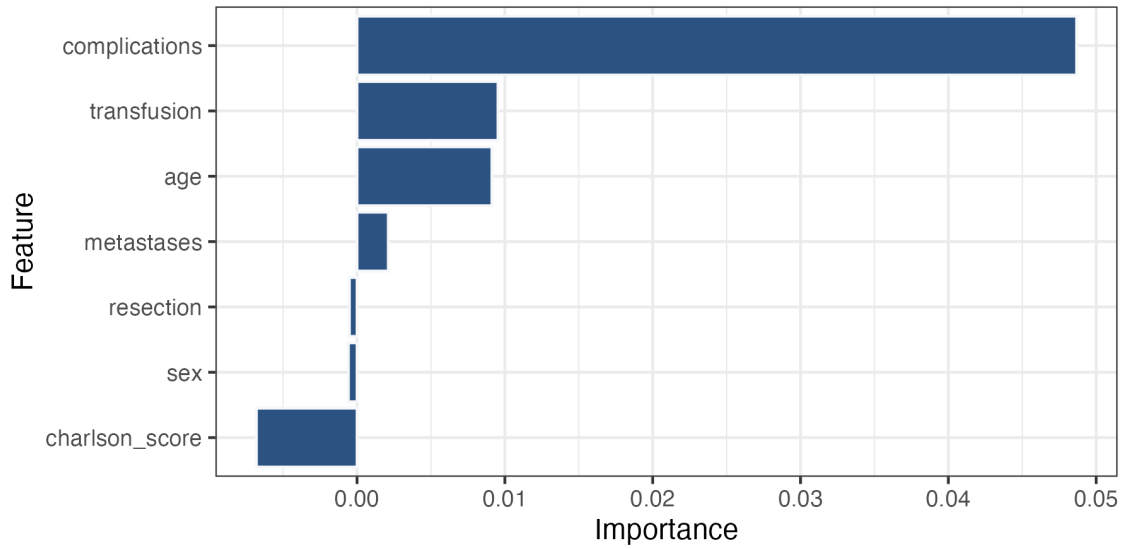

Figure 5: Variable Importance for all features of the tumor dataset.

3.5 Handling missing data

For classical decision trees, and therefore also for random forests, several approaches exist to handle missing data. For CART the idea of surrogate splits was introduced (Leo Breiman et al. 1984). The idea is to replace the primary split by an alternative split whenever the splitting feature is missing. If the best split at a node is given by the feature x^* and the split point c^*, a surrogate split is a split based on another feature x and a split point c that comes closest to the primary split. If x^* is missing for an observation, the surrogate split is used to assign the observation to one of the child nodes. However, this method can be computationally expensive, especially in Random Forests, where many trees are grown and surrogate splits would have to be computed repeatedly at many nodes. Therefore, Ishwaran, Kogalur, et al. (2008) propose a new algorithm to handle missing features for random forests. The algorithm works as follows (Ishwaran, Kogalur, et al. 2008, p. 13):

1. Imagine we have nodes $1, ..., H$ to split. Before splitting we impute the data in each of the H nodes. For each node h and feature x_p we define a set of non-missing values $X_{h,p}$ and a set of missing values $X^*_{h,p}$. We impute all missing values by drawing from the empirical distribution of $X_{h,p}$. Then we start splitting the nodes, as we have no missing data left.

2. In the resulting child nodes we reset the imputed data to missing and repeat Step 1.

3. Proceed until the tree can no longer be split.

An advantage of this algorithm is the fact that the OOB data remain untouched during the imputation procedure, in contrast to the proximity approach (Breiman 2003). So the OOB prediction error is not optimistically biased.

4 Evaluation

Evaluation is an important part of the machine learning workflow. It allows us to assess how well a model works with unseen test data and is the basis for comparing different

models on the same data. For our Machine Learning tasks with survival data, we need specific evaluation techniques. Since we estimate an unknown function in the learners we derived in section 2 and section 3, we cannot simply compare the predicted values with the true values like the MSE [6] does for example. In this section, we will introduce two measures which are suited for survival data and our prediction targets.

4.1 Concordance Index (C-index)

The first measure is Harrell's concordance index which was introduced by Harrell et al. (1982). It measures the ability of a model to rank individuals correctly. The idea is that observations with earlier observed events have a higher predicted risk than observations that experience the event later. As one might expect, this is done by comparing pairs of observations. For this, we first need to define a set of comparable pairs.
Two observations i and j with $T_i < T_j$, which means i had the event before j, are comparable if the earlier observation in our case i corresponds to an observed event, that is, if $\delta_i = 1$, as seen in Figure 6.

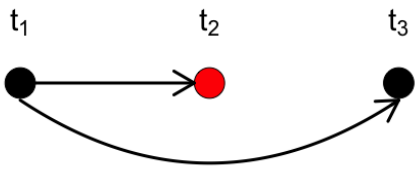

Figure 6: Comparable pairs for the C-index. Illustration adapted from Wang, Li, and Reddy (2019)

Observation t_1 has the earliest event time and the event is observed. Therefore, t_1 can be compared with both t_2 and t_3. In contrast, t_2 is censored and can therefore only be compared with the earlier observation t_1. It cannot be compared with t_3, since it is unknown whether the event for t_2 would have occurred before or after t_3.
Once the set of comparable pairs has been defined, each pair can be evaluated by comparing the corresponding predicted risks. For each comparable pair, there are three possible outcomes:

- concordant pair: the observation with the earlier event has a higher predicted risk;
- discordant pair: the observation with the earlier event has a lower predicted risk;
- tied pair: both observations have the same predicted risk.

The three possible outcomes are illustrated in Figure 7.

[6] $\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\text{truth}_i - \text{predicted}_i)^2$

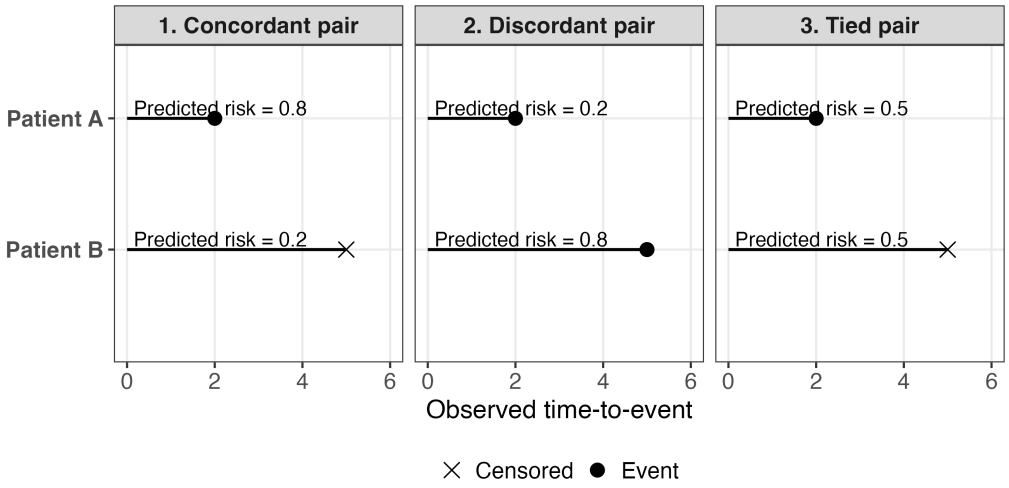

Figure 7: Possible outcomes for comparable pairs in the C-index

The C-index is then calculated as follows:

$$\hat{C} = \frac{|\text{concordant pairs}| + 0.5 \times |\text{tied pairs}|}{|\text{comparable pairs}|}. \tag{5}$$

The interpretation of (5) is similar to that of the AUC.[7] A value of $\hat{C} = 1$ indicates a perfect ranking of risks. In other words, the model orders every comparable pair of observations correctly. A value of $\hat{C} = 0.5$ indicates that the model ranks the observations no better than random guessing, since a random ranking has probability $\frac{1}{2}$ of ordering a comparable pair correctly. The C-index evaluates ranking, not the absolute accuracy of predicted survival probabilities.
One possible application of the C-index is in settings where resources are limited, such as hospitals with restricted treatment capacities. In such situations, a model that accurately ranks patients according to their mortality risk can be useful for identifying high-risk patients and assigning them higher priority.
Now, we want to discuss how we can use the C-Index to evaluate a Random-Survival-Forest which we discussed in section 3. After fitting the random forest each observation receives a scalar risk score $\hat{r}_i$. A common choice for the risk score is ensemble mortality (Ishwaran, Kogalur, et al. 2008, p. 6). Remember the prediction of the Random Survival Forest was the function in (4) which is a function of time t. The ensemble mortality for observation i is just the sum over all time points:

$$\hat{r}_i = \hat{M}_{ens}(\mathbf{x}_i) = \sum_{j=1}^{m} \hat{H}(t_j \mid \mathbf{x}_i)$$

Higher values of $\hat{r}_i$ correspond to higher predicted risk. Hence, for two observations i and j with $T_i < T_j$ and $\delta_i = 1$, we aim to obtain $\hat{r}_i > \hat{r}_j$.

4.2 Brier score

In subsection 4.1 we discussed a measure which evaluates how well a model discriminates. Now, we want to introduce a measure that evaluates the accuracy of predictions made by a survival model.
The Brier score is a measure for rating the accuracy of probabilistic prediction (Glenn

[7] See Hanley and McNeil (1982).

et al. 1950). Let $\hat{\pi}_i(t) \in [0,1]$ denote a predicted probability at time t and $c_i(t) \in \{0,1\}$ denote the true outcome at time t. Then the Brier score at time t is given by:

$$\widehat{\mathrm{BS}}(t) = \frac{1}{N}\sum_{i=1}^{N}\left[\hat{\pi}_i(t) - c_i(t)\right]^2.$$

The score was introduced to evaluate weather forecasts. It can be interpreted as a mean squared error for binary prediction targets.
Graf et al. (1999) extended this idea to survival outcomes. The Brier score using inverse probability of censoring weights (IPCW) is defined by:

$$\widehat{\mathrm{BS}}(t) = \frac{1}{N}\sum_{i=1}^{N}\left[\frac{\mathbb{I}(T_i \leq t, \delta_i = 1)}{\hat{G}(T_i)}\hat{S}_i(t)^2 + \frac{\mathbb{I}(T_i > t)}{\hat{G}(t)}(1-\hat{S}_i(t))^2\right], \tag{6}$$

where $G(\tau) = \mathbb{P}[C_i > \tau]$ is the probability that an observation is not censored up to time τ. $\hat{G}$ is estimated using the Kaplan-Meier estimator on the observations $(T_i, 1-\delta_i)$.
The main idea of IPCW is to compensate for the information loss due to censoring. Observations that remain uncensored in regions where censoring is likely receive larger weights, so that they also represent similar observations that were censored earlier. For example, suppose that $\hat{G}(\tau) = 0.5$. This means that the estimated probability of not being censored up to time τ is 0.5. Hence, only about half of the observations are expected to remain uncensored up to this time point. The uncensored observations therefore receive the weight

$$\frac{1}{\hat{G}(\tau)} = \frac{1}{0.5} = 2.$$

Intuitively, each uncensored observation represents itself and, on average, one similar observation that was censored earlier.
In (6) we can distinguish three different cases:

- Event observed before t: we compare the survival prediction with survival status 0
- Observed beyond t: we compare the prediction with survival status 1
- Censored before t: no direct contribution to (6). The observation still contributes indirectly, since it is used in the estimation of the censoring distribution $\hat{G}$.

Notice that $\widehat{BS}(t)$ is time-dependent. In other words, it is a function of time t. We can visualize the time-dependent prediction error by plotting $\widehat{\mathrm{BS}}(t)$ over the evaluation time interval. This is shown in Figure 8. Here we plot the performance of a Random Survival Forest fitted on the `tumor` data. The black line is the $\widehat{BS}(t)$ of the Kaplan-Meier estimator that serves as a reference model. We can see the performance of both models at different time points. The Random Survival Forest has for every time point a better performance than the reference model.

The IPCW Brier score evaluates the accuracy of survival predictions. One application for the IPCW Brier score could be in unemployment duration modeling. Here, the event of interest may be re-employment. In this case, the survival function describes the probability that an individual remains unemployed beyond a given time point. Accurate probability estimates are very important for planning and allocating resources, such as financial support.

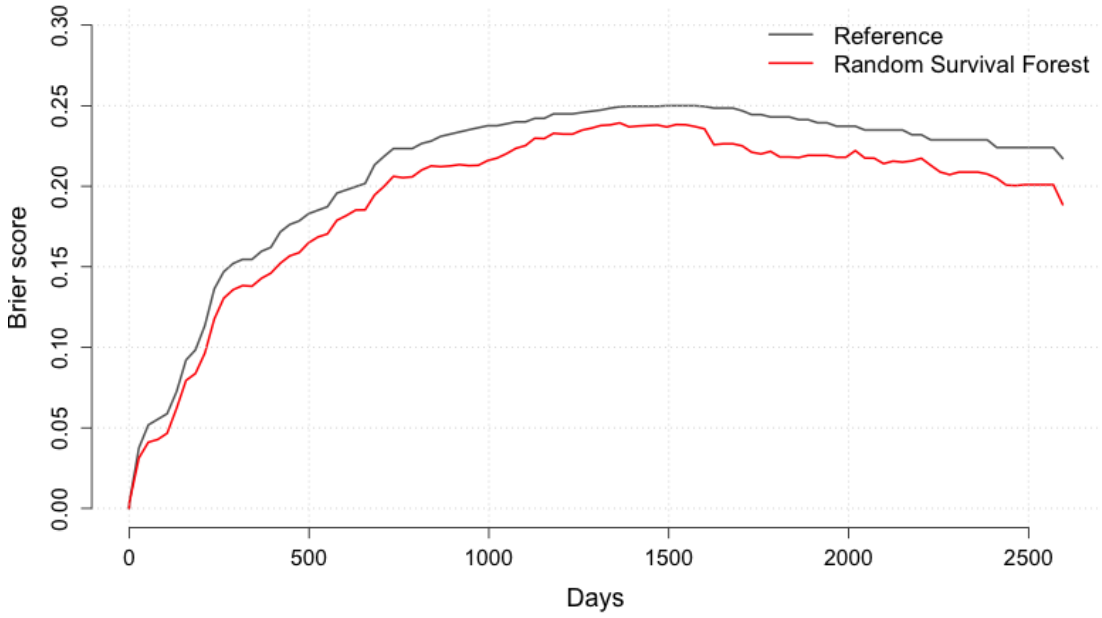

Figure 8: IPCW Brier score for the tumor data over time

4.3 Integrated Brier score

The IPCW Brier score from subsection 4.2 was a function of time t. To obtain a single scalar performance measure, we consider the integrated Brier score (IBS), which summarizes the prediction error over a given time interval. It is defined as the average $\widehat{BS}$ over the interval $[t_{\min}, t_{\max}]$ ((Park et al. 2021, p. 1705)):

$$\widehat{\text{IBS}} = \frac{1}{t_{\max} - t_{\min}} \int_{t_{\min}}^{t_{\max}} \widehat{\text{BS}}\, dt$$

Lower values of $\widehat{\text{IBS}}$ indicate better prediction accuracy.

5 Application

In this section, we apply our derived methods from section 3 and section 4 to real data. For this purpose, we conduct a small benchmark study where we compare the Random-Survival Forest with two classical statistical approaches to model survival data. First, we take the Kaplan-Meier estimator, see (2), as a non-informative baseline model without using covariates.
Our second model is the Cox proportional hazards model, introduced in Cox (1972). It is a semi-parametric regression model that relates the hazard rate to a vector of covariates $\mathbf{x}_i$ while leaving the baseline hazard unspecified. For an individual with covariate vector $\mathbf{x}$, the hazard function is given by:

$$h(t \mid \mathbf{x}) = h_0(t) \exp(\mathbf{x}^\top \beta), \tag{7}$$

where $h_0(t)$ denotes the baseline hazard and β the vector of regression coefficients. The model in (7) relies on the proportional hazards assumption, which means the hazard ratio between two individuals is constant over time. Formally, for two observations with

covariate vectors $\mathbf{x}_1$ and $\mathbf{x}_2$, this ratio is given by:

$$\frac{h(t \mid \mathbf{x}_1)}{h(t \mid \mathbf{x}_2)} = \frac{\exp(\mathbf{x}_1^\top \beta)}{\exp(\mathbf{x}_2^\top \beta)},$$

which is independent of time. A second important assumption is the functional form of the covariate effects. Furthermore, in the standard form no interaction effects between covariates are included unless they are specified explicitly.

We now compare these two classical statistical approaches with our Random Survival Forest which is a machine learning approach. In contrast to the Cox model, the Random Survival Forest does not rely on the proportional hazards assumption, does not require a prespecified functional form for the covariate effects, and can capture nonlinear effects and interactions between covariates automatically.

For the Random Survival Forest, we consider two different variants. First, we fit an untuned Random Survival Forest using a fixed set of default hyperparameters. Second, we fit a tuned Random Survival Forest, where the hyperparameters `mtry` and d_0 are selected by a tuning procedure on the training data. This allows us to investigate whether adapting the model complexity to the data improves predictive performance compared to the untuned version. In both cases, we train the forest with $B = 500$ trees. The tuning procedure is carried out using random search (Yang and Shami 2020, p. 303) with 30 evaluations. Random search samples candidate hyperparameter configurations from predefined search spaces and evaluates their performance using an inner resampling procedure. In our case, the search spaces are given by

$$\texttt{mtry.ratio} = \frac{\texttt{mtry}}{p} \in \{0.1, 0.2, \ldots, 1.0\} \quad \text{and} \quad d_0 \in \{3, 4, \ldots, 50\},$$

where p denotes the number of covariates. For the inner resampling, we use 5-fold cross-validation on the training data. The hyperparameter configuration with the best average validation performance is selected and used to refit the tuned Random Survival Forest on the complete training set. The tuning criterion is the integrated Brier score from subsection 4.3.

For the outer resampling and evaluation of the four models we use repeated 5-fold cross validation with 3 repetitions. As evaluation metrics we use the C-index described in subsection 4.1 and the integrated Brier score. We apply the whole procedure to two different datasets. First, the already used `tumor` dataset. The head of the dataset can be seen in Table 2. The other one is a dataset of breast cancer patients which can be found in the `R`-package `survival` (Therneau 2024) under the name `gbsg`. The first four observations of

Table 3: Head of survival::gbsg dataset

rfstime	status	pid	age	meno	size	grade	nodes	pgr	er	hormon
1838	0	132	49	0	18	2	2	0	0	0
403	1	1575	55	1	20	3	16	0	0	0
1603	0	1140	56	1	40	3	3	0	0	0
177	0	769	45	0	25	3	1	0	4	0

the data are shown in Table 3 and a short summary of both datasets is given in Table 4. The results of the benchmark for the c-index are given in Figure 9. We can see that for both datasets the Kaplan-Meier estimator has a value of 0.5. This fact is intuitive, since the Kaplan–Meier estimator returns the same survival function for every observation, so that all comparable pairs are tied pairs. If we insert this into (5), we obtain:

$$\hat{C} = \frac{0.5 \times |\text{comparable pairs}|}{|\text{comparable pairs}|} = 0.5,$$

Table 4: Summary of datasets

Dataset	Observations	Features	censoring rate
gbsg	686	8	0.564
tumor	776	7	0.517

since we have no concordant pairs and all comparable pairs are tied. Furthermore, we can see that the two forests perform slightly better than the Cox model on the `gbsg` dataset. For the `tumor` data set, the Cox model achieves the highest C-index. However, the values of the Cox model and the two forests are all in a similar range.
The results of the integrated Brier score can be seen in Figure 10. For both datasets the Kaplan-Meier estimator has the worst performance. Apart from that, the results are similar to those for the C-index. It is only important to note that, as described in subsection 4.3, lower values are better in this case.
In summary, the results of the Cox model and the Random Survival Forest do not differ substantially on the data considered in this benchmark. One possible explanation is that the data sets contain only a limited number of features and observations (see Table 4). Therefore, the underlying structure may already be captured well by the Cox model. In this setting, the additional flexibility of the Random Survival Forest does not lead to a clear improvement in predictive performance. The tuning procedure also did not lead to a substantial improvement over the untuned Random Survival Forest. This may indicate that the default hyperparameters were already reasonably well suited to the data. Alternatively, the considered tuning space and the number of random search evaluations may have been too limited to identify substantially better configurations. Since the observed performance differences are relatively small, no clear superiority of one method over the other can be concluded from this benchmark alone.

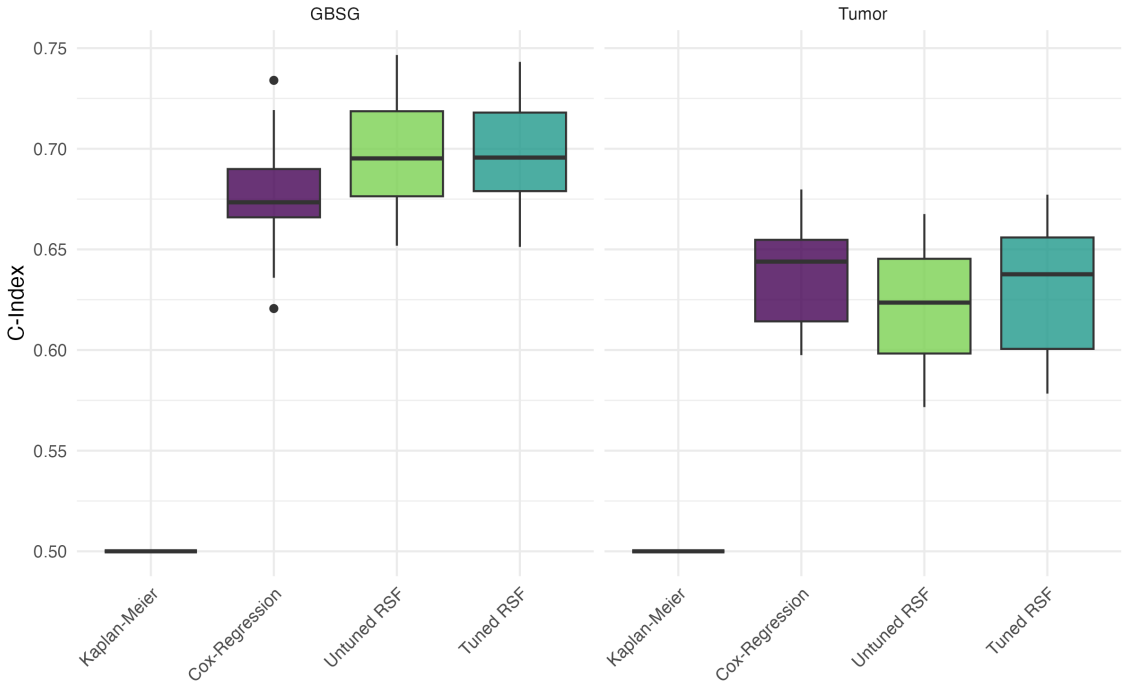

Figure 9: Benchmark results with the C-index

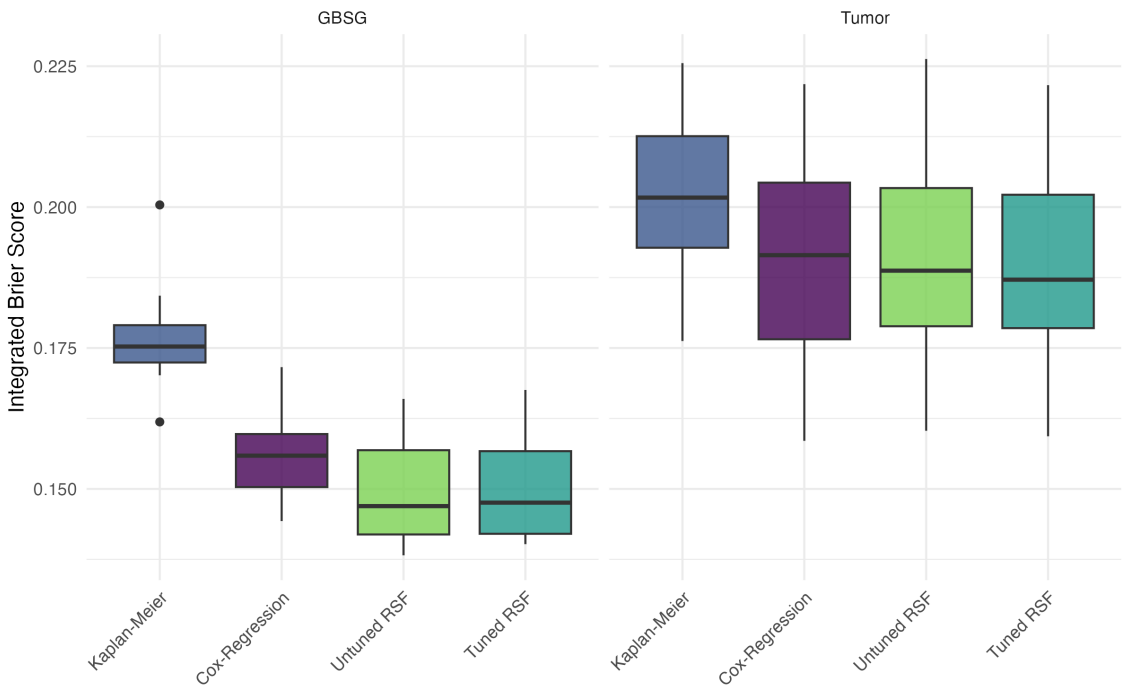

Figure 10: Benchmark results with the integrated Brier score

The benchmark was implemented in `R` (R Core Team 2025). The `mlr3` ecosystem (Lang et al. 2019) was used, including `mlr3proba` (Sonabend et al. 2021) and `mlr3tuning` (Becker et al. 2025).

6 Discussion

In this seminar paper, two machine learning approaches for modeling survival outcomes were introduced. First, we discussed survival trees, which extend classical decision trees to right-censored survival data. We showed how survival trees differ from ordinary decision trees. While classical decision trees use splitting criteria designed for regression or classification tasks, survival trees require splitting rules that account for censored time-to-event outcomes. For that we used the log-rank splitting rule, which selects the feature and split point that maximize the separation between the resulting survival distributions in the child nodes. Another difference is the prediction target. Instead of predicting outcomes or probabilities, survival trees estimate functions like the survival function or the cumulative hazard function.
We then extended this idea to obtain a more stable learner, since survival trees like ordinary decision trees can be quite unstable. For this reason, we introduced the Random Survival Forest. Similar to a classical Random Forest, that learner grows many individual trees based on bootstrap samples drawn from the original data. By aggregating the prediction of these trees, the resulting predictions become more stable and the variance is reduced. In the following, we discuss both advantages and limitations of this machine learning approach.

Advantages A first advantage of the Random Survival Forest is that it does not rely on the proportional hazards assumption, unlike the Cox model in (7). Moreover, it does not require a prespecified functional form for the feature effects. Instead, the relationship between the features and the survival outcome is learned in a data-driven way. In addition, interactions between features can be captured automatically through the recursive splitting

structure of the trees. Furthermore, we are flexible in our predicted property. We can either predict the survival function or the cumulative hazard function. Another advantage is the internal validation procedure based on the OOB prediction error, as well as the internal variable importance method. As a final point, there is also an algorithm that automatically handles missing data.

Limitations On the other hand, Random Survival Forests also have some limitations. One limitation is that the effect of individual covariates is not directly interpretable in contrast to the Cox model. In the Cox model from (7), the regression coefficients can be interpreted through hazard ratios. For example, suppose that we have a numerical covariate x. If x is increased by one unit, while all other covariates are held constant, the corresponding hazard ratio is given by:

$$\frac{h(t \mid x+1, \mathbf{z})}{h(t \mid x, \mathbf{z})} = \frac{h_0(t)\exp((x+1)\beta + \mathbf{z}^\top \boldsymbol{\gamma})}{h_0(t)\exp(x\beta + \mathbf{z}^\top \boldsymbol{\gamma})} = \exp(\beta).$$

Thus, $\exp(\beta)$ describes the multiplicative change in the hazard associated with a one-unit increase in x. Such a simple interpretation is not available for Random Survival Forests. Another limitation of Random Survival Forests is their computational cost. At each node, the algorithm has to evaluate possible split points for the candidate features. Although only a random subset of features is considered at each split, this can still be computationally demanding, especially for numerical features with many possible split points. The computational burden increases further when the forest is combined with hyperparameter tuning, since many different model configurations have to be fitted and evaluated. However, as observed in section 5, tuning did not lead to a substantial improvement in predictive performance for the two data sets considered in this seminar paper. This suggests that, at least in these applications, the additional computational cost of tuning may not be justified by a clear gain in performance. A final limitation concerns the log-rank splitting rule. Although Random Survival Forests do not rely on the proportional hazards assumption in the same way as the Cox model, the log-rank test is most powerful when the hazards are approximately proportional. If this assumption is violated, for example when survival curves cross or covariate effects change over time, the power of the log-rank test may decrease. As a consequence, a split that would separate the node well in terms of survival behavior may receive only a small log-rank statistic and therefore may not be selected.
Furthermore, we considered the evaluation of machine learning models in survival analysis. For this purpose, we introduced one measure that assesses the ranking of the predictions, namely the C-index, and another measure that evaluates the accuracy of the predicted survival probabilities, namely the Brier score and its integrated version, the integrated Brier score.

A Electronic Appendix

GitHub repository: `https://github.com/JakobHaas999/SurvForML`
This repository contains the R code used for generating the figures and conducting the empirical analysis. The repository contains a `README.md` file that describes the structure of the repository and provides instructions for reproducing the analysis.

References

Aalen, Odd (1978). "Nonparametric inference for a family of counting processes". In: *The Annals of Statistics*, pp. 701–726.

Becker, Marc et al. (2025). *mlr3tuning: Hyperparameter Optimization for 'mlr3'*. R package version 1.4.0. DOI: 10.32614/CRAN.package.mlr3tuning. URL: https://CRAN.R-project.org/package=mlr3tuning.

Bender, Andreas and Fabian Scheipl (2018). "pammtools: Piece-wise exponential Additive Mixed Modeling tools". In: *arXiv:1806.01042 [stat]*. URL: https://arxiv.org/abs/1806.01042.

Breiman, L (2003). *Manual-setting up, using, and understanding random forests v4. 0. Using random forests v4. 0. pdf.*

Breiman, Leo et al. (1984). *Classification and Regression Trees*. Belmont, CA: Wadsworth.

Cox, David R (1972). "Regression models and life-tables". In: *Journal of the royal statistical society: Series B (methodological)* 34.2, pp. 187–202.

Glenn, W Brier et al. (1950). "Verification of forecasts expressed in terms of probability". In: *Monthly weather review* 78.1, pp. 1–3.

Graf, Erika et al. (1999). "Assessment and comparison of prognostic classification schemes for survival data". In: *Statistics in medicine* 18.17-18, pp. 2529–2545.

Hanley, James A and Barbara J McNeil (1982). "The meaning and use of the area under a receiver operating characteristic (ROC) curve." In: *Radiology* 143.1, pp. 29–36.

Harrell, Frank E et al. (1982). "Evaluating the yield of medical tests". In: *Jama* 247.18, pp. 2543–2546.

Ishwaran, Hemant and Udaya B Kogalur (2007). "Random survival forests for R". In: *R news* 7.2, pp. 25–31.

Ishwaran, Hemant, Udaya B Kogalur, et al. (2008). "Random survival forests". In.

James, Gareth et al. (2013). *An introduction to statistical learning: with applications in R*. Vol. 103. Springer.

Kaplan, Edward L and Paul Meier (1958). "Nonparametric estimation from incomplete observations". In: *Journal of the American statistical association* 53.282, pp. 457–481.

Lang, Michel et al. (Dec. 2019). "mlr3: A modern object-oriented machine learning framework in R". In: *Journal of Open Source Software*. DOI: 10.21105/joss.01903. URL: https://joss.theoj.org/papers/10.21105/joss.01903.

Mantel, Nathan et al. (1966). "Evaluation of survival data and two new rank order statistics arising in its consideration". In: *Cancer Chemother Rep* 50.3, pp. 163–170.

Nelson, Wayne (1972). "Theory and applications of hazard plotting for censored failure data". In: *Technometrics* 14.4, pp. 945–966.

Park, Seo Young et al. (2021). "Review of statistical methods for evaluating the performance of survival or other time-to-event prediction models (from conventional to deep learning approaches)". In: *Korean Journal of Radiology* 22.10, p. 1697.

R Core Team (2025). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. Vienna, Austria. URL: https://www.R-project.org/.

Sonabend, Raphael et al. (Feb. 2021). "mlr3proba: An R Package for Machine Learning in Survival Analysis". In: *Bioinformatics*. ISSN: 1367-4803. DOI: 10.1093/bioinformatics/btab039.

Therneau, Terry M (2024). *A Package for Survival Analysis in R*. R package version 3.8-3. URL: https://CRAN.R-project.org/package=survival.

Wang, Ping, Yan Li, and Chandan K. Reddy (2019). "Machine Learning for Survival Analysis: A Survey". In: *ACM Computing Surveys* 51.6, pp. 1–36.

Yang, Li and Abdallah Shami (2020). "On hyperparameter optimization of machine learning algorithms: Theory and practice". In: *Neurocomputing* 415, pp. 295–316.

Use of AI

I hereby confirm that I wrote all substantive parts of this paper myself. In addition, I used **ChatGPT 5.5 Thinking** as a tool, specifically to obtain linguistically refined formulations in English. I also used **ChatGPT 5.5 Thinking** to check the seminar paper for typos and language errors. I used **Codex** to adjust axis labels and legends of figures in some cases.